KUBERNETES

COMPLETE NOTES

(Cloud & DevOps Engineering)



BEGINNER TO ADVANCED

12 Topics | Diagrams + YAML | kubectl Cheat Sheets | Interview Ready

handwritten-style notes ~ simplify, revise, practise

Anas Kadambalath

TOPICS COVERED

Tip!
Practise every command on minikube, kind or k3d as you read.

A one-stop, beginner-to-advanced revision guide for Cloud & DevOps engineers ~ concepts, diagrams, YAML and kubectl, topic by topic.

What's inside

1. **Kubernetes Fundamentals** — 3
   what K8s is, why it exists, desired state, key features
2. **Architecture & Core Components** — 4
   control plane, worker nodes, what happens on kubectl apply
3. **Pods, ReplicaSets & Deployments** — 5
   workload hierarchy, probes, requests & limits
4. **Rolling Updates & Rollbacks** — 6
   maxSurge, maxUnavailable, rollout commands, canary & blue-green
5. **Services & Ingress** — 7
   ClusterIP, NodePort, LoadBalancer, DNS, L7 routing
6. **Namespaces, Labels & Selectors** — 8
   isolation, quotas, grouping and matching objects
7. **ConfigMaps & Secrets** — 9
   externalising config, consuming as env or files, securing secrets
8. **Volumes, PV, PVC & StorageClass** — 10
   ephemeral vs persistent storage, dynamic provisioning
9. **StatefulSets, DaemonSets, Jobs & CronJobs** — 11
   choosing the right workload controller
10. **Scheduling, Affinity, Taints & Tolerations** — 12
    how the scheduler picks a node, and how to steer it
11. **Security ~ RBAC, Service Accounts & Network Policies** — 13
    who can do what, pod identity, pod-level firewalls
12. **Essential kubectl Commands & End-to-End Workflow** — 14-15
    cheat sheet, troubleshooting, code to production

Colour code

- Red words ~ key terms to remember
- Highlighted ~ most-asked in interviews
- `monospace` ~ YAML fields & commands
- Yellow notes ~ tips, gotchas & best practices

Practice playground

- **minikube / kind / k3d** ~ local clusters
- **Killercoda** ~ free browser-based labs
- **EKS / AKS / GKE** ~ managed cloud clusters
- **kubernetes.io/docs** ~ the official reference

1. KUBERNETES FUNDAMENTALS

Remember!
K8s = K + 8 letters + s. The name is Greek for helmsman (pilot).

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, networking and management of containerized applications across a cluster of machines.

What is Kubernetes?

- Runs and manages containers across many machines (a cluster) as if they were one big computer.
- Groups containers into logical units called Pods for easy management & discovery.
- Declarative: you describe *what* you want in YAML; K8s works out *how*.
- Born at Google (inspired by Borg), open-sourced in 2014; now run by the CNCF.



Why Kubernetes?

- To manage containers at scale
- To automate deployment & scaling
- To keep apps highly available
- To use resources efficiently
- To stay portable: cloud, on-prem, hybrid

Docker vs Kubernetes

	Docker Engine	Kubernetes
Job	Build & run images	Orchestrate containers
Scope	Single host	Cluster of many nodes
Scaling	Manual	Manual or auto (HPA)
Healing	Restart policy only	Replaces failed Pods
Networking	Per-host bridges	Cluster network + DNS

Not rivals: Docker builds images, K8s runs them via containerd.

Desired State ~ the core idea

You write YAML (desired state) → API Server (stores in etcd) → Controllers (watch & act)

observe → compare → fix the difference ↺

This never-ending reconciliation loop is why K8s can self-heal: actual state is always pushed back to desired state.

Key Features

1	Automated Rollouts	Deploys new versions gradually and rolls back if something breaks.
2	Self-Healing	Restarts crashed containers, replaces & reschedules failed Pods.
3	Horizontal Scaling	Scale replicas up or down by command, or automatically on CPU/memory.
4	Service Discovery & LB	Gives Pods a stable DNS name and spreads traffic across them.
5	Storage Orchestration	Mounts local, cloud or network storage automatically.
6	Config & Secret Mgmt	Changes config and credentials without rebuilding images.
7	Bin Packing	Places Pods on nodes based on CPU/memory requests to save resources.
8	Extensibility	Add your own resources & controllers with CRDs and Operators.

Key Terms

Cluster	Set of machines running K8s
Node	A worker machine (VM or physical)
Pod	Smallest deployable unit (1+ containers)
Control Plane	The brain that makes decisions
kubectl	CLI used to talk to the cluster

Benefits

- ★ Faster, safer & repeatable releases
- ★ Better resource utilisation & lower cost
- ★ Zero-downtime updates & high availability
- ★ Same setup across dev, test & prod
- ★ No vendor lock-in: runs on any cloud

2. ARCHITECTURE & CORE COMPONENTS

Interview! etcd holds ALL cluster state. Lose etcd = lose the cluster. Back it up!

A Kubernetes cluster = Control Plane (the brain that *decides*) + Worker Nodes (the muscle that *runs* your Pods). Every component talks through the API Server.

Cluster Architecture



You + **kubectl**

→

CONTROL PLANE (manages the cluster)

kube-apiserver	etcd	kube-scheduler	controller-manager	cloud-controller
front door	key-value DB	picks a node	reconciles	talks to cloud

⇅ kubelets watch the API Server & report status (HTTPS) ⇅

Worker Nodes (run apps)

Worker Node 1: kubelet | kube-proxy | containerd



Worker Node 2: kubelet | kube-proxy | containerd



Control Plane Components

kube-apiserver	Front door: validates every request; the **only** component that talks to etcd.
etcd	Distributed key-value store holding all cluster state.
kube-scheduler	Picks the best node for every new, unassigned Pod.
controller-mgr	Runs controllers (Node, ReplicaSet, Job…) that keep actual = desired.
cloud-ctrl-mgr	Links K8s to cloud APIs: load balancers, routes, nodes.

Worker Node Components

kubelet	Node agent: keeps its Pods' containers running & healthy, reports status.
kube-proxy	Programs iptables / IPVS rules so Service traffic reaches Pods.
Runtime	Pulls images & runs containers via CRI: containerd, CRI-O.
Pods	Your application containers.

dockershim was removed in v1.24 ~ Docker-built images still run fine.

What happens when you run `kubectl apply -f deploy.yaml` ?

1. **kubectl** sends the YAML to the API Server.
2. API Server: authN → authZ (RBAC) → admission.
3. The Deployment object is saved in etcd.
4. Controller creates a ReplicaSet → Pods.
5. Scheduler binds each new Pod to a node.
6. kubelet has containerd pull & start it.
7. Status reported; Service endpoints updated.
8. Controllers keep watching → self-healing.

Essential Add-ons

CoreDNS	Cluster DNS: service names → IPs
CNI plugin	Pod networking: Calico, Cilium, Flannel
Metrics Server	CPU/RAM for `kubectl top`, HPA
Ingress Controller	HTTP(S) routing into the cluster

Interview Nuggets

- HA = 3 or 5 control-plane nodes (etcd quorum).
- Only the API Server writes to etcd.
- EKS, AKS, GKE manage the control plane.
- Components use *watch* on the API, not polling.

3. PODS, REPLICASETS & DEPLOYMENTS

Remember! You rarely create Pods by hand ~ let a Deployment manage them.

A Pod is the smallest deployable unit. A ReplicaSet keeps N identical Pods running. A Deployment manages ReplicaSets to give you declarative updates & rollbacks.

The Hierarchy

Deployment (web) → ReplicaSet (web-7d9f8c) → 3 Pods (app + sidecar container) → container (your image runs here)

rollouts & scaling — keeps replicas = 3

Pod

- Wraps 1+ containers that share one IP (talk via `localhost`) and volumes.
- Ephemeral: a dead Pod is replaced, not repaired (new name & IP).
- Init containers run first; sidecars add logging, proxies.

ReplicaSet

- Ensures the desired number of replicas is always running.
- Finds its Pods with a label selector.
- Replaces deleted / failed Pods automatically.
- Normally created *by* a Deployment.

Deployment

- Declarative rolling updates & rollbacks.
- Easy scaling up / down.
- Keeps revision history (old ReplicaSets).
- Default choice for stateless apps.

Deployment YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}    # must match ↓
  template:                    # Pod template
    metadata:
      labels: {app: web}
    spec:
      containers:
      - name: web
        image: nginx:1.27
        ports: [{containerPort: 80}]
        resources:
          requests: {cpu: 100m, memory: 128Mi}
          limits:   {cpu: 500m, memory: 256Mi}
        readinessProbe:
          httpGet: {path: /, port: 80}
        livenessProbe:
          httpGet: {path: /, port: 80}
```

Health Probes

Probe	Asks...	If it fails
liveness	Is it alive?	Container restarted
readiness	Ready for traffic?	Removed from Service
startup	Finished booting?	Other probes wait

Check types: `httpGet`, `tcpSocket`, `exec`, `grpc`

Pod Lifecycle

Pending → Running → Succeeded / Failed

Common errors: CrashLoopBackOff (keeps crashing), ImagePullBackOff (can't pull image), OOMKilled (over memory limit), Pending (no node fits).

Requests, Limits & QoS

- requests = reserved amount, used by the scheduler.
- limits = hard cap. CPU over limit → *throttled*; memory over limit → *OOMKilled*.
- QoS: Guaranteed req = limit, Burstable some set, BestEffort none ~ evicted first.

Commands

```
$ kubectl create deploy web --image=nginx --replicas=3
$ kubectl get deploy,rs,pods -o wide
$ kubectl scale deploy web --replicas=5
$ kubectl describe pod <pod-name>
$ kubectl delete pod <pod-name> # RS recreates
$ kubectl explain pod.spec.containers
```

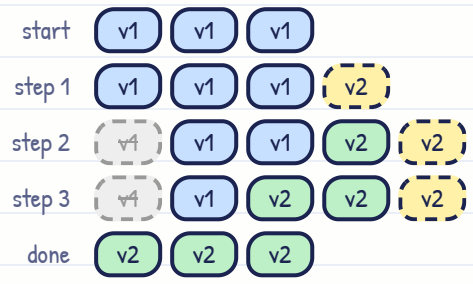
4. ROLLING UPDATES & ROLLBACKS

Gotcha! No readiness probe? K8s may send traffic to Pods that aren't ready yet.

A rolling update replaces old Pods with new ones a few at a time, so the app stays available. A rollback returns the Deployment to an earlier revision.

How a Rolling Update Works

Example: `replicas: 3`, `maxSurge: 1`, `maxUnavailable: 0` ~ change image `v1` → `v2`



Strategy Types

RollingUpdate (default)	Gradual swap, zero downtime. v1 & v2 run side by side briefly.
Recreate	Kill all old, then start new. **Downtime**, but no version mix.

maxSurge & maxUnavailable

- maxSurge: extra Pods allowed *above* replicas. Default **25%**.
- maxUnavailable: Pods allowed to be *unavailable*. Default **25%**.
- Number or %; both can't be 0. surge 1 / unavailable 0 = safest.

Strategy YAML

```yaml
spec:
  replicas: 4
  minReadySeconds: 10
  revisionHistoryLimit: 10
  progressDeadlineSeconds: 600
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

progressDeadlineSeconds: marks a stuck rollout as failed.

Rollout Commands

`kubectl set image deploy/web web=nginx:1.27`	update
`kubectl rollout status deploy/web`	watch it
`kubectl rollout history deploy/web`	revisions
`kubectl rollout history deploy/web --revision=2`	details
`kubectl rollout undo deploy/web`	undo
`kubectl rollout undo deploy/web --to-revision=2`	go to rev 2
`kubectl rollout pause | resume deploy/web`	batch edits
`kubectl rollout restart deploy/web`	fresh Pods

Record why: `kubectl annotate deploy/web kubernetes.io/change-cause="nginx 1.27"` (`--record` is deprecated)

What triggers a new revision? Only changes to the Pod template (`.spec.template`): image, env, labels, resources... Scaling replicas does NOT create a new revision.

Advanced Strategies

Blue-Green	Run v1 & v2 fully, then flip the Service selector. Instant switch & rollback.
Canary	Small % of traffic to v2, watch metrics, then increase.
Tools	Argo Rollouts, Flagger, Istio / Linkerd

Best Practices

- Always define readiness probes
- Use exact image tags, never `:latest`
- Set `minReadySeconds` to catch early crashes
- Add a PodDisruptionBudget for HA
- Test the rollback path in staging first

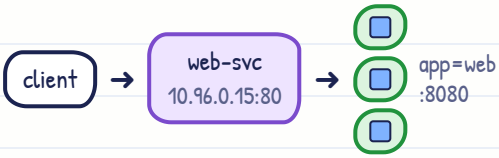
5. SERVICES & INGRESS

Remember! Service = stable address for changing Pods. Ingress = smart HTTP router.

A Service gives a group of Pods one stable IP & DNS name and load-balances traffic across them. Ingress routes external HTTP/HTTPS traffic to Services by *host* and *path*.

Why Services?

- Pods are ephemeral: they die, move and get **new IPs**.
- A Service finds its Pods using a label selector.
- Healthy Pod IPs are tracked in EndpointSlices; kube-proxy routes traffic to them.



Service YAML & Ports

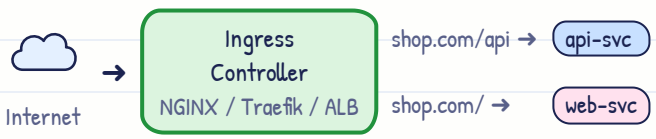
```yaml
apiVersion: v1
kind: Service
metadata: {name: web-svc}
spec:
  type: ClusterIP       # default
  selector: {app: web}  # Pod labels
  ports:
  - port: 80            # Service port
    targetPort: 8080    # container port
```

nodePort = port opened on every node (NodePort type)

Service Types

Type	What it does	Use it for
ClusterIP (default)	Internal virtual IP, reachable only inside the cluster	Service-to-service traffic
NodePort	Opens a static port `30000–32767` on every node	Dev, testing, bare metal
LoadBalancer	Provisions an external cloud load balancer	Exposing apps on EKS / AKS / GKE
ExternalName	DNS CNAME to an outside name, no proxying	External DB / API alias
Headless	`clusterIP: None` ~ DNS returns the Pod IPs directly	StatefulSets, client-side LB

Ingress ~ Layer 7 Routing



- Routes by host & path: one entry for many Services.
- TLS termination using a TLS Secret.
- Saves cost: one load balancer instead of many.
- Needs an Ingress Controller ~ the Ingress object alone does nothing!

Gateway API ~ the newer, role-based successor to Ingress: GatewayClass → Gateway → HTTPRoute, with traffic splitting & header matching built in.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: {name: shop}
spec:
  ingressClassName: nginx
  tls:
  - {hosts: [shop.com], secretName: shop-tls}
  rules:
  - host: shop.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            {name: api-svc, port: {number: 80}}
```

Service DNS

`<svc>.<ns>.svc.cluster.local`

- e.g. `web-svc.prod.svc.cluster.local`
- Same namespace? Just `http://web-svc`
- Resolved by CoreDNS

Commands

```
$ kubectl expose deploy web --port=80
$ kubectl get svc,endpointslices,ing
$ kubectl port-forward svc/web-svc 8080:80
$ kubectl describe ingress shop
```

6. NAMESPACES, LABELS & SELECTORS

app=web

Gotcha! Namespaces isolate names, NOT traffic ~ use NetworkPolicies.

Namespaces split one cluster into virtual clusters. Labels are key=value tags on objects. Selectors find objects by their labels ~ the glue that connects Services, ReplicaSets & Pods.

Namespaces

- Isolate teams, environments or apps in one cluster.
- Names are unique *within* a namespace, not across.
- Scope for RBAC, ResourceQuota & LimitRange.
- Deleting a namespace deletes everything inside!

default	Where objects go if you don't say
kube-system	Control plane & add-ons
kube-public	Readable by everyone (cluster-info)
kube-node-lease	Node heartbeats (Lease objects)

ResourceQuota caps the namespace's *total* usage; LimitRange sets default / min / max requests & limits *per container*.

Namespaced vs Cluster-scoped

Namespaced:

Pod · Deployment · Service · ConfigMap · Secret · PVC · Role · ServiceAccount

Cluster-scoped:

Node · Namespace · PV · StorageClass · ClusterRole · CRD

Check: `kubectl api-resources --namespaced=false`

ResourceQuota

```
kind: ResourceQuota
metadata: {name: dev-quota, namespace: dev}
spec:
  hard: {pods: "20", requests.cpu: "4",
         requests.memory: 8Gi, limits.memory: 16Gi}
```

Labels & How Selectors Connect Things

- Labels = key/value pairs in `metadata.labels`: `app=web`, `env=prod`, `tier=frontend`.
- Many labels per object; used to group, select & filter.
- Recommended: `app.kubernetes.io/name`, `/version`, `/component`, `/part-of`, `/managed-by`.



Selector Types

Type	Operators	Example
Equality-based	`=` `==` `!=`	`kubectl get pods -l env=prod,tier!=cache`
Set-based	`in` `notin` `exists`	`kubectl get pods -l 'env in (prod,staging)'`
In YAML	`matchLabels`, `matchExpressions`	`{key: env, operator: In, values: [prod]}`

Commands

```
$ kubectl create namespace dev
$ kubectl get pods -n dev   # -A = all ns
$ kubectl config set-context --current \
    --namespace=dev
$ kubectl label pod web-1 env=prod
$ kubectl label pod web-1 env-  # remove
$ kubectl get pods --show-labels -L env
```

Labels vs Annotations

	Labels	Annotations
Purpose	Select objects	Extra metadata
Selectable?	Yes	No
Values	Short (≤ 63)	Can be large
Examples	app, env, tier	change-cause

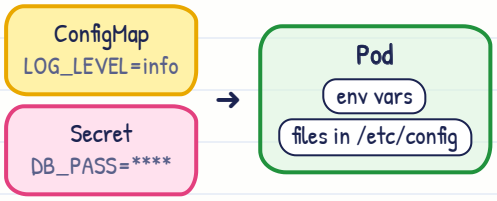
7. CONFIGMAPS & SECRETS

> **Warning!** base64 is encoding, NOT encryption. Anyone who can read a Secret can decode it.

ConfigMaps hold non-sensitive configuration; Secrets hold sensitive data (passwords, tokens, keys). Both keep config out of your container image.

Why separate config?

- Build once, run anywhere: same image for dev, test, prod
- Change settings without rebuilding images
- Follows the 12-factor app principle



ConfigMap vs Secret

	ConfigMap	Secret
For	Non-sensitive config	Sensitive data
Stored as	Plain text	base64-encoded
Max size	1 MiB	1 MiB
Examples	LOG_LEVEL, URLs, config files	DB password, API key, TLS cert
On node	Normal files	tmpfs (memory) when mounted

Create Them

```
$ kubectl create configmap app-config --from-literal=LOG_LEVEL=info --from-file=app.properties
$ kubectl create secret generic db-secret --from-literal=DB_PASS='S3cr3t!'
$ kubectl create secret tls web-tls --cert=tls.crt --key=tls.key
$ kubectl create secret docker-registry regcred --docker-server=<registry> --docker-username=<user> ...
$ echo -n 'S3cr3t!' | base64          # UzNjcjN0IQ==
$ echo 'UzNjcjN0IQ==' | base64 -d      # S3cr3t!  ← see? not encrypted
```

Use Them in a Pod

```
spec:
  containers:
  - name: app
    image: myapp:1.0
    env:                          # 1) one key
    - name: DB_PASS
      valueFrom:
        secretKeyRef: {name: db-secret, key: DB_PASS}
    envFrom:                      # 2) all keys
    - configMapRef: {name: app-config}
    volumeMounts:                 # 3) as files
    - {name: cfg, mountPath: /etc/config}
  volumes:
  - name: cfg
    configMap: {name: app-config}
```

Secret Types

Opaque	Default, any key/value
kubernetes.io/tls	TLS cert + key
.../dockerconfigjson	Private registry login
.../basic-auth	Username + password
.../ssh-auth	SSH private key

Env vars vs Volumes

- Env vars: read at start ~ **not updated** on change (restart Pod).
- Volume files: refreshed automatically after a short delay (not for `subPath`).

Keeping Secrets Safe

- Enable encryption at rest for etcd (KMS)
- Restrict `get` / `list` on secrets with RBAC
- Never commit secrets to Git
- External stores: Vault, AWS Secrets Manager, Azure Key Vault via External Secrets Operator / CSI driver
- Sealed Secrets for GitOps; `immutable: true` for fixed config

8. VOLUMES, PV, PVC & STORAGECLASS

Remember! PV = the disk. PVC = the request. StorageClass = the disk factory.

Files inside a container vanish when it restarts. Volumes give Pods storage; PersistentVolumes keep data beyond the life of any Pod.

How Persistent Storage Fits Together

Pod (mounts /data) → PVC ("I need 10Gi, RWO") → binds PV (10Gi volume) ← StorageClass (creates PVs on demand)

The PV is backed by real storage (AWS EBS, Azure Disk, GCE PD, NFS). With a StorageClass this is dynamic provisioning ~ no admin ticket needed.

PV vs PVC vs StorageClass

Object	What it is	Created by	Scope
PersistentVolume	A piece of real storage available to the cluster	Admin (static) or SC (dynamic)	Cluster
PersistentVolumeClaim	A request for storage: size + access mode (+ class)	Developer	Namespace
StorageClass	Template for creating PVs on demand: provisioner, parameters, reclaim policy	Admin	Cluster

Inspect: `kubectl get pv,pvc,sc` ~ `kubectl describe pvc data-pvc`

Volume Types

emptyDir	Scratch space; deleted with the Pod
hostPath	Node folder ~ **avoid in prod**
configMap/secret	Config injected as files
PVC	Durable storage through a PV
CSI drivers	EBS, Azure Disk, GCE PD, Ceph

Access Modes

RWO	ReadWriteOnce	read-write by **one node**
ROX	ReadOnlyMany	read-only by many nodes
RWX	ReadWriteMany	read-write by many nodes
RWOP	ReadWriteOncePod	read-write by **one Pod**

Block disks (EBS, Azure Disk) are usually RWO; RWX needs shared storage like NFS, EFS or Azure Files.

Dynamic Provisioning YAML

```yaml
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata: {name: fast-ssd}
provisioner: ebs.csi.aws.com
parameters: {type: gp3}
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
---
kind: PersistentVolumeClaim
apiVersion: v1
metadata: {name: data-pvc}
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: fast-ssd
  resources: {requests: {storage: 10Gi}}
--- # in the Pod spec
volumes:
- name: data
  persistentVolumeClaim: {claimName: data-pvc}
```

Reclaim Policy

Retain	Data kept after PVC delete; manual cleanup
Delete	PV + disk deleted (dynamic default)
Recycle	Deprecated ~ don't use

PV status: Available → Bound → Released

WaitForFirstConsumer creates the disk only after the Pod is scheduled, so it lands in the same zone as the node.

9. StatefulSets, DaemonSets, Jobs & CronJobs

Quick pick!
DB? StatefulSet
Per node? DaemonSet
Run once? Job

Beyond Deployments, Kubernetes has specialised workload controllers for stateful apps, per-node agents and run-to-completion batch tasks.

StatefulSet

- Stable names: `db-0`, `db-1`, `db-2`
- Stable DNS via a headless Service: `db-0.db`
- Own PVC per Pod via `volumeClaimTemplates`
- Ordered: start 0 → N, scale down N → 0

db-0 + disk → db-1 + disk → db-2 + disk

Use: MySQL, PostgreSQL, MongoDB, Kafka, Elasticsearch

DaemonSet

- Runs one Pod on every node (or selected nodes)
- New node joins → Pod added automatically
- Node removed → its Pod is cleaned up
- Target nodes with `nodeSelector` / tolerations

node-1 [agent] node-2 [agent] node-3 [agent]

Use: Fluent Bit, node-exporter, CNI plugins, kube-proxy

Job

- Runs Pods until the task succeeds, then stops
- `completions` (how many) & `parallelism` (at once)
- `backoffLimit` = retries (default 6)
- `ttlSecondsAfterFinished` = auto-cleanup
- `restartPolicy`: Never / OnFailure

Use: DB migrations, batch processing, one-off scripts

CronJob

- Creates Jobs on a schedule (cron format)
- `concurrencyPolicy`: Allow | Forbid | Replace
- Keeps last 3 successful / 1 failed Job by default
- `timeZone` sets the zone; `suspend: true` pauses it

Use: backups, reports, cleanup, cert renewal

Cron Syntax

*/15	2	*	*	1-5
minute 0-59	hour 0-23	day of month	month 1-12	weekday 0=Sun

`0 2 * * *`	Every day at 02:00
`*/5 * * * *`	Every 5 minutes
`0 9 * * 1`	Mondays at 09:00
`0 0 1 * *`	1st of every month

CronJob YAML

```yaml
apiVersion: batch/v1
kind: CronJob
metadata: {name: db-backup}
spec:
  schedule: "0 2 * * *"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      backoffLimit: 3
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - {name: backup, image: backup-tool:1.4}
```

Which Controller When?

	Deployment	StatefulSet	DaemonSet	Job / CronJob
Pod names	Random hash	Ordered: app-0, app-1	One per node	Random hash
Storage	Shared / none	Own PVC per Pod	Usually node-local	Temporary
Scaling	replicas	replicas, in order	= node count	parallelism
Runs	Forever	Forever	Forever	Until done

```
$ kubectl get sts,ds,jobs,cronjobs
$ kubectl scale sts db --replicas=5
$ kubectl create job hi --image=busybox -- echo hi
$ kubectl create job now --from=cronjob/db-backup
```

10. SCHEDULING, AFFINITY, TAINTS & TOLERATIONS

Analogy! Taint = a 'Keep out' sign on a node. Toleration = the Pod's access pass.

The kube-scheduler picks the best node for every new Pod. You steer its choice with nodeSelector, affinity rules and taints & tolerations.

How the Scheduler Decides

New Pod (no node yet) → **1. Filter** drop nodes that can't run it: resources, taints, selectors → **2. Score** rank the rest: spread, affinity, balance → **3. Bind** best node wins; its kubelet starts the Pod

No node passes? Pod stays Pending ~ `kubectl describe pod` shows e.g. *"0/3 nodes are available: insufficient cpu"*.

Ways to Steer Placement

Method	How it works	Typical use
nodeSelector	Pod runs only on nodes with matching labels	Simple: `disktype=ssd`
Node affinity	Rich node rules, **required** (hard) or **preferred** (soft)	Zones, instance types, GPUs
Pod (anti-)affinity	Place near / away from Pods with certain labels	Co-locate cache; spread replicas
Topology spread	Keeps Pod counts even across zones / nodes (`maxSkew`)	High availability
PriorityClass	Higher-priority Pods can **pre-empt** (evict) lower ones	Critical system workloads

Node Affinity & Pod Anti-Affinity

```yaml
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - {key: disktype, operator: In, values: [ssd]}
    preferredDuringSchedulingIgnoredDuringExecution:
    - weight: 50
      preference:
        matchExpressions:
        - {key: gpu, operator: Exists}
```

```yaml
affinity:
  podAntiAffinity:  # 1 web Pod per node
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchLabels: {app: web}
      topologyKey: kubernetes.io/hostname
# topologyKey: kubernetes.io/hostname = node
#          topology.kubernetes.io/zone = zone
```

required = must ~ preferred = try (weight 1–100) ~ IgnoredDuringExecution = running Pods stay if labels change. Ops: `In NotIn Exists DoesNotExist Gt Lt`

Taints & Tolerations

Pod A ✓ tolerates gpu → gpu-node (taint gpu=true:NoSchedule)

Pod B ✗ no toleration ⇥ gpu-node (taint gpu=true:NoSchedule)

```yaml
tolerations:
- key: gpu
  operator: Equal
  value: "true"
  effect: NoSchedule
```

NoSchedule	New Pods without a toleration aren't placed
PreferNoSchedule	Scheduler tries to avoid the node
NoExecute	Also **evicts** running Pods that don't tolerate it

```
$ kubectl taint nodes node1 gpu=true:NoSchedule
$ kubectl taint nodes node1 gpu=true:NoSchedule-
$ kubectl label nodes node1 disktype=ssd
```

Node maintenance (cordon / drain) → see Topic 12

Tolerations **allow**, they don't **attract**! To dedicate nodes, combine taint + toleration + node affinity. Control-plane nodes carry `node-role.kubernetes.io/control-plane:NoSchedule`.

11. SECURITY

RBAC, Service Accounts & Network Policies

Golden rule! Least privilege ~ grant only what's needed, nothing more.

Kubernetes security is layered: RBAC controls who can call the API, Service Accounts give Pods an identity, and Network Policies control which Pods can talk to each other.

The 4C's of Cloud-Native Security

- **Cloud** ~ IAM, network, provider security
  - **Cluster** ~ RBAC, API access, NetworkPolicies
    - **Container** ~ trusted images, non-root
      - **Code** ~ deps, secrets, TLS

Life of an API Request

1. Authentication ~ *who are you?* certs, tokens, OIDC
2. Authorisation ~ *are you allowed?* RBAC
3. Admission control ~ *is it valid & compliant?* Pod Security, quotas, policies
4. Saved to etcd ✓

Fail any step → `401` / `403` / request rejected.

RBAC ~ Role-Based Access Control

Role	Permissions inside **one namespace**
ClusterRole	Cluster-wide, or for nodes / PVs
RoleBinding	Grants a Role to subjects in a namespace
ClusterRoleBinding	Grants a ClusterRole across the cluster

Subject (app-sa) → RoleBinding → Role (get, list pods)

- Subjects: User, Group, ServiceAccount
- Verbs: get, list, watch, create, update, patch, delete
- RBAC is additive only ~ there are no "deny" rules.

```
$ kubectl auth can-i list pods -n dev \
    --as=system:serviceaccount:dev:app-sa
```

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata: {name: pod-reader, namespace: dev}
rules:
- apiGroups: [""]          # core API group
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
---
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata: {name: read-pods, namespace: dev}
subjects:
- {kind: ServiceAccount, name: app-sa, namespace: dev}
roleRef: {kind: Role, name: pod-reader,
          apiGroup: rbac.authorization.k8s.io}
```

Network Policies ~ a Firewall for Pods

frontend → ✓ :8080 → backend

other Pod → ✗

- By default every Pod can reach every Pod.
- Once a policy selects a Pod, only allowed traffic gets in.
- Match by `podSelector`, `namespaceSelector`, `ipBlock`.
- Needs a CNI that enforces it: Calico, Cilium.
- **Default deny**: `podSelector: {}` + no rules.

```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata: {name: allow-frontend, namespace: prod}
spec:
  podSelector:                  # protects...
    matchLabels: {app: backend}
  policyTypes: [Ingress]
  ingress:
  - from:                       # ...allows only
    - podSelector:
        matchLabels: {app: frontend}
    ports: [{protocol: TCP, port: 8080}]
```

Service Accounts

- Identity for apps in Pods (users are for humans).
- Every namespace has a `default` SA.
- Pods get a short-lived, auto-rotated token.
- Set `automountServiceAccountToken: false` if unused.
- Cloud IAM: EKS Pod Identity, Workload Identity.

`kubectl create sa app-sa -n dev` → `spec.serviceAccountName: app-sa`

Hardening Checklist

- ✓ Pod Security: Baseline / Restricted
- ✓ `runAsNonRoot`, read-only root FS, drop capabilities
- ✓ Scan images (Trivy) & use trusted registries
- ✓ Encrypt Secrets at rest; turn on audit logs
- ✓ Policy engines: Kyverno, OPA Gatekeeper
- ✓ Keep Kubernetes versions patched

12. ESSENTIAL KUBECTL COMMANDS

Shortcut!
alias k=kubectl and turn on shell autocompletion.

kubectl is the CLI for the Kubernetes API ~ create, inspect, debug and manage every resource.

Command Syntax

```
kubectl   get   pods   web-1   -n dev -o wide
```

kubectl	get	pods	web-1	-n dev -o wide
the CLI	command	resource TYPE	NAME (optional)	flags

Cluster & Context

```
kubectl version                # client + server
kubectl cluster-info           # API & DNS URLs
kubectl get nodes -o wide
kubectl config get-contexts    # list clusters
kubectl config use-context prod
kubectl api-resources          # all types
```

Create & Apply

```
kubectl apply -f app.yaml      # create/update
kubectl apply -f ./k8s/        # whole folder
kubectl diff -f app.yaml       # preview
kubectl create deploy web --image=nginx
kubectl expose deploy web --port=80
kubectl run tmp -it --rm --image=busybox -- sh
```

View & Inspect

```
kubectl get pods -o wide       # node + IP
kubectl get all -n dev
kubectl get pod web-1 -o yaml
kubectl describe pod web-1     # + events
kubectl get events -n dev
kubectl explain deploy.spec    # field docs
```

Debug

```
kubectl logs web-1 -f          # stream
kubectl logs web-1 --previous  # last crash
kubectl logs web-1 -c sidecar
kubectl exec -it web-1 -- sh   # shell
kubectl port-forward svc/web 8080:80
kubectl top pods               # CPU / memory
kubectl debug -it web-1 --image=busybox
```

Modify & Scale

```
kubectl scale deploy web --replicas=5
kubectl autoscale deploy web --min=2 --max=10
kubectl set image deploy/web web=nginx:1.27
kubectl edit deploy web        # edit live
kubectl label pod web-1 env=prod
kubectl annotate deploy web owner=team-a
```

Nodes & Cleanup

```
kubectl cordon node1           # no new Pods
kubectl drain node1 --ignore-daemonsets
kubectl uncordon node1         # back in service
kubectl delete -f app.yaml
kubectl delete pod web-1       # RS recreates
kubectl delete all -l app=web
```

Handy Flags

`-n dev` / `-A`	one / all namespaces
`-o wide | yaml | json`	output format
`-l app=web`	filter by label
`-w`	watch for changes
`--dry-run=client -o yaml`	generate YAML without creating

Short Names

po · deploy · rs · svc · ns · no · cm · sa · pv · pvc · sc · sts · ds · cj · ing · netpol · hpa · ep

e.g. `kubectl get po,svc -n dev`

Imperative vs Declarative

	Imperative ~ you say *how*	Declarative ~ you say *what* (YAML)
Commands	`create`, `run`, `expose`, `scale`	`apply -f` (+ Git, Helm, Argo CD)
Best for	Quick tests, learning, exams	Production: repeatable, reviewable, versioned

12. END-TO-END DEPLOYMENT WORKFLOW

Pro tip! Start manifests with `--dry-run=client -o yaml`

From source code to a running, exposed & monitored app ~ the path every DevOps engineer should know.

The Workflow

1. **Write code** — app + Dockerfile
2. **Build image** — docker build
3. **Push to registry** — Docker Hub / ECR / ACR
4. **Write manifests** — Deploy, Svc, Ingress, CM, Secret
5. **kubectl apply** — or Helm / Argo CD
6. **Verify rollout** — status, pods, events
7. **Expose & test** — Service, Ingress, port-forward
8. **Monitor & scale** — metrics, logs, HPA
9. **Update / rollback** — set image, rollout undo

Step-by-Step Commands

```
# 1-3  build & push
$ docker build -t myrepo/shop:1.0 .
$ docker push myrepo/shop:1.0
# 4  namespace, config & secret
$ kubectl create ns shop
$ kubectl create configmap shop-cfg \
    --from-literal=LOG_LEVEL=info -n shop
$ kubectl create secret generic shop-db \
    --from-literal=DB_PASS='***' -n shop
# 5  deploy everything in ./k8s
$ kubectl apply -f k8s/ -n shop
# 6  verify
$ kubectl rollout status deploy/shop -n shop
$ kubectl get pods,svc,ing -n shop
# 7  test locally
$ kubectl port-forward svc/shop 8080:80 -n shop
# 8  autoscale
$ kubectl autoscale deploy shop --max=10 -n shop
# 9  update, and roll back if needed
$ kubectl set image deploy/shop \
    shop=myrepo/shop:1.1 -n shop
$ kubectl rollout undo deploy/shop -n shop
```

Troubleshooting Cheat Sheet

Symptom	Check first	Usual cause
Pending	`describe pod` → Events	Not enough CPU/memory, taints, unbound PVC
ImagePullBackOff	`describe pod`	Wrong image / tag, missing imagePullSecret
CrashLoopBackOff	`logs --previous`	App error, bad config, failing liveness probe
OOMKilled	`describe pod` → Last State	Memory limit too low, or a leak
Running, not Ready	`describe pod`	Readiness probe failing
Service no response	`get endpointslices`	Selector ≠ Pod labels, wrong targetPort

Production-Ready Checklist

- [x] Probes + requests & limits on every container
- [x] Pinned image tags, never `:latest`
- [x] HPA + PodDisruptionBudget
- [x] Ingress with TLS; NetworkPolicies
- [x] Own ServiceAccount + least-privilege RBAC

CI/CD & GitOps

git push → CI: build, test, scan → push image → Argo CD / Flux sync

- GitOps: Git is the single source of truth; the cluster syncs itself.
- Package with Helm or Kustomize; rollback = `git revert`.

Keep going: Helm, Kustomize, Operators & CRDs, service mesh (Istio / Linkerd), Prometheus + Grafana ~ then aim for CKA, CKAD or CKS.